

[2팀]

개선 할 수 있는 추가사항

1. service, repository layer에서 반환 타입 명시해 주기

- Response 용 DTO를 구현 해 두었다면

```
export const getProduct = async (dto: GetProductDTO): Promise<ProductResponseDTI
⇒ {
  //로직 구현
}
```

2. async 함수는 언제나 Promise를 반환 한다.

repository layer

```
// await 붙였을때,
export async function getById(id: number): Promise<Comment | null> {
  // 1) prismaClient.comment.findUnique()가 Promise<Comment|null>을 반환
  // 2) await가 그 Promise를 "풀어(unwrapping)"서 실제 Comment|null 값을 얻음
  // 3) return 값(예: Comment|null)을 다시 Promise.resolve()로 감싸서 반환
  return await prismaClient.comment.findUnique({ where: { id } });
}
```

```
// async - await 생략한 경우
export function getById(id: number): Promise<Comment | null> {
  // 1) prismaClient.comment.findUnique()가 Promise<Comment|null>을 반환
  // 2) 함수가 그 Promise를 그대로 반환
  return prismaClient.comment.findUnique({ where: { id } });
}
```

- await이 있을때,
 1. `findUnique()` 호출 → Promise 얻음
 2. `await` 로 값(`Comment|null`)을 꺼냄
 3. 그 값을 다시 `async` 함수가 자동으로 `Promise.resolve()` 감싸서 반환

- await 생략 시
 1. `findUnique()` 호출 → Promise 얻음
 2. 그대로 반환



await 생략 시 얻을 수 있는 장점

1. 코드가 간결
2. await가 많으면 에러가 발생했을 때, stack trace가 복잡해 질 수 있음
 - a. Promise를 그대로 반환하면 호출부에서 발생한 에러를 그대로 잡아 낼 수 있어 디버깅 명확
3. 미세한 성능차이 → 극단적으로 호출했을때 찾아볼 수 있음 (await선언시 Microtask 생성)

await을 꼭 사용해야 할 때?

1. 가져온 값으로 가공을 해야할때
2. try-catch로 에러를 잡을때,

`prismaClient.xxx()` 가 반환하는 Promise를 그대로 반환만 한다면, `await` 를 생략해도 동작상·타입상 전혀 차이가 없습니다. 코드를 더 깔끔하게 유지하기 위해 불필요한 `await` 는 과감히 제거하는 걸 추천

`await` 키워드는 **Promise**가 해결(resolve)된 후의 값을 바로 사용하고 싶을 때 사용
하지만 함수가 단순히 다른 Promise를 그대로 반환만 하고, 그 결과값을 추가로 가공하거나 `try/catch` 로 잡아서 처리할 필요가 없다면, `await` 를 생략해도 전혀 문제가 없음.

`prismaClient.xxx()` ⇒ 비동기 함수 이고 반환 값이 Promise임

- `async-await` 없이 작성되어도 여전히 `Promise<Comment | null>` 을 반환하므로 서비스레이어에서 해당 함수를 호출 할 때, `await` 을 통해 호출

```
// 레포지토리 레벨
export function getById(id: number): Promise<Comment | null> {
  return prismaClient.comment.findUnique({ where: { id } });
}

// 서비스 레이어
export async function getCommentById(id: number) {
```

```
// getById가 Promise<Comment | null>를 반환하므로 await을 사용하여 실제 값을 받아옴
const comment = await getById(id);

return comment;
}
```

Package.json script 작성

```
// Prisma용 script도 작성하는것을 추천
"scripts": {
  "prestart": "npm run build",
  "start": "node dist/app.js",
  "dev": "nodemon --watch src --exec ts-node src/app.ts",
  "build": "tsc",
  "prisma:migrate": "prisma migrate dev",
  "prisma:generate": "prisma generate",
  "prisma:seed": "ts-node prisma/seed.ts",
},
```

<https://www.mysqltutorial.org/getting-started-with-mysql/>

raw 쿼리를 잘 설계하면 → orm 을 사용하는데 더 쉬움

필요에 따라 테이블 Indexing explain을 통한 쿼리 실행계획 → 성능비교

집계에 탁월한 window함수 같은것도 있는것 참조

mcp

<https://www.youtube.com/watch?v=jpWtV08fF-E&t=11s>